



UNIVERSIDAD POLITÉCNICA DE PACHUCA
PROGRAMA EDUCATIVO DE INGENIERÍA EN SOFTWARE
ASIGNATURA: INTELIGENCIA ARTIFICIAL

EXPLICACIÓN

Fundamentación Teórica Exhaustiva, Modelado OLS, Diagnósticos Gauss-Markov, Aprendizaje UCB1 y Guía de Código Paso a Paso

Información Institucional y Académica

Autores:	Emmanuel Islas Lozada Ximena Nathaly Angeles Sanchez
Grado y Grupo:	8º Cuatrimestre – Ingeniería en Software
Asignatura:	Inteligencia Artificial
Proyecto:	Sistema Inteligente de Despacho mediante RL y OLS (UPP Route RL)
Fecha de Entrega:	Agosto 2026
Ubicación:	Pachuca de Soto, Hidalgo, México

Índice

1. Introducción Pedagógica y Diagramas de Arquitectura	3
1.1. El Dilema del Transporte Universitario	3
1.2. Diagrama de Flujo Comparativo: Tradicional vs. Adaptativo UCB1	4
2. Diccionario Ilustrado del Dataset	5
2.1. Esquema Relacional entre Variables	5
3. Teoría de Probabilidad y Proceso Estocástico de Poisson	7
3.1. Deducción Matemática Rigurosa de la Fórmula de Poisson	7
3.1.1. Explicación Término por Término de la Fórmula	8
3.2. Comportamiento Dinámico de la Tasa $\lambda(t)$ por Franja Horaria	8
4. Modelado Estadístico por Regresión Lineal OLS (Mínimos Cuadrados Ordinarios)	9
4.1. ¿Qué es OLS y Por Qué se Eligió?	9
4.2. Deducción Paso a Paso por Cálculo Diferencial de las Ecuaciones Normales .	9
4.2.1. Paso 1: Derivada Parcial respecto al Intercepto $\hat{\beta}_0$	9
4.2.2. Paso 2: Derivada Parcial respecto a la Pendiente $\hat{\beta}_1$	10
4.3. Resultados Empíricos del Modelo y Graficación	10
4.4. Explicación del Coeficiente $R^2 = 0.121$	11
4.4.1. ¿Por qué es 12,1 % y por qué es el mejor resultado posible para este problema?	11
5. Pruebas de Supuestos de Gauss-Markov y Diagnósticos	13
5.1. Supuesto 1: Linealidad en Parámetros	13
5.2. Supuesto 2: Independencia de Residuos (Prueba de Durbin-Watson)	13
5.3. Supuesto 3: Homocedasticidad (Prueba de Breusch-Pagan)	13
6. Aprendizaje por Refuerzo Prescriptivo y Algoritmo UCB1	15
6.1. Diagrama del Ciclo Agente-Entorno en UCB1	15
6.2. Deducción y Explicación de la Fórmula UCB1	15



6.2.1.	Desglose Técnico Término por Término	15
6.3.	Función de Recompensa Normalizada R_t	16
7.	Guía Exhaustiva de Código Fuente Paso a Paso	17
7.1.	Módulo 1: <code>src/config.py</code> – Parámetros y Constantes de Sistema	17
7.1.1.	Explicación Línea por Línea	18
7.2.	Módulo 2: <code>src/data_generator.py</code> – Generador Estocástico Poisson	18
7.2.1.	Explicación Línea por Línea	19
7.3.	Módulo 3: <code>src/regression_model.py</code> – Ajuste OLS y Diagnósticos	20
7.3.1.	Explicación Línea por Línea	20
7.4.	Módulo 4: <code>src/ucb_bandit.py</code> – Agente de Aprendizaje por Refuerzo	21
7.4.1.	Explicación Línea por Línea	21
8.	Guía Rápida de Defensa Académica y Banco de Preguntas	23

1. Introducción Pedagógica y Diagramas de Arquitectura

1.1. El Dilema del Transporte Universitario

El servicio de transporte colectivo para la Universidad Politécnica de Pachuca (UPP) enfrenta una contradicción estructural entre dos objetivos contrapuestos:

1. **Criterio Económico del Chofer/Operador:** Minimizar el consumo de combustible y maximizar la rentabilidad saliendo únicamente cuando la unidad está llena al 100 % (18/18 pasajeros).
2. **Criterio de Calidad del Estudiante:** Minimizar el tiempo de espera en la bahía para no perder clases, reduciendo tiempos muertos que en horas valle pueden superar los 60 minutos.

La política tradicional rígida obliga a la combi a esperar hasta llenar los 18 asientos sin importar el tiempo transcurrido. En horas pico (de 06:00 a 09:00 y de 11:00 a 13:00 hrs), esto funciona eficientemente porque la tasa de arribo es alta ($\lambda = 3,5$ alumnos/min) y la combi se llena en 3 a 5 minutos. Sin embargo, en horas valle ($\lambda = 1,2$ alumnos/min), esperar 18 alumnos genera tiempos de espera excesivos de 45 a 60 minutos, colapsando la satisfacción estudiantil.

1.2. Diagrama de Flujo Comparativo: Tradicional vs. Adaptativo UCB1

El siguiente esquema (Figura 1) ilustra la diferencia entre la política tradicional rígida y la solución adaptativa prescriptiva desarrollada con Aprendizaje por Refuerzo (UCB1).

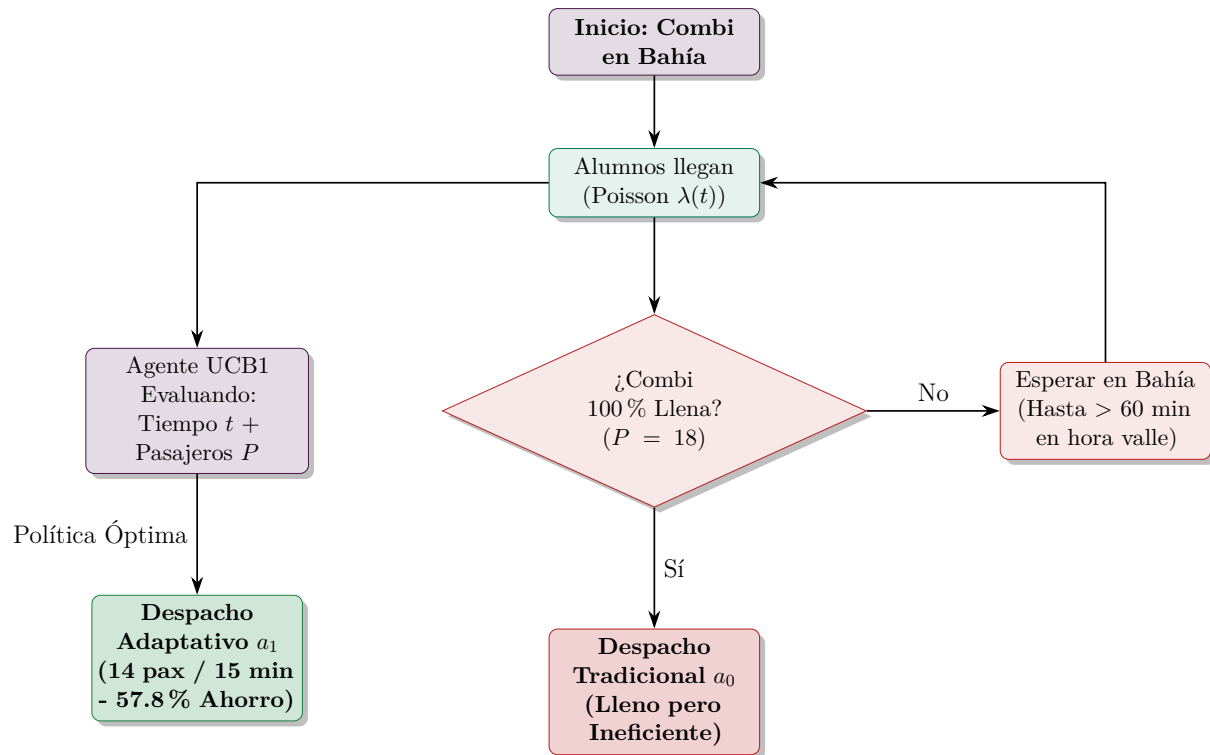


Figura 1: Esquema Comparativo: Flujo Rígido Tradicional vs. Flujo Inteligente UCB1.

2. Diccionario Ilustrado de Campos del Dataset

El dataset oficial de la simulación contiene $n = 1,920$ registros correspondientes a despachos y estados de cola durante 21 días laborables (3 semanas de operación real).

Tabla 1: Diccionario Exhaustivo Campo por Campo del Dataset Oficial

Campo del Dataset	Tipo	Propósito Técnico y Conexión con el Modelo
fecha	Date	Fecha de la observación (YYYY-MM-DD) a lo largo de 21 días.
hora	String	Hora exacta del primer arribo de alumno a la fila (HH:MM).
ruta	Factor	Ruta operada (<i>Las Vías</i> o <i>Puente Tuzos</i>).
alumnos_esperando	Integer	Variable Independiente X en OLS. Alumnos en cola al llegar la combi a la bahía.
llegadas_intervalo	Integer	Arribos estocásticos de alumnos simulados mediante Poisson $\lambda(t)$ durante el tiempo de abordaje.
hora_salida	String	Hora exacta HH:MM en que la combi enciende motor y parte de la bahía.
pasajeros_al_salir	Integer	Pasajeros sentados a bordo ($P \leq 18$). Alimenta la recompensa RL.
tipo_salida	Factor	Tipo de evento: <i>Llena</i> ($P = 18$), <i>Parcial</i> ($P < 18$) o <i>Ninguna</i> .
tiempo_espera_acum	Float	Variable Dependiente Y en OLS. Minutos acumulados de espera en bahía.
tiempo_recorrido	Float	Minutos transcurridos en trayecto hasta la UPP.
paradas_intermed	Integer	Número de paradas secundarias realizadas en carretera (0 a 4).
alumnos_recogidos	Integer	Alumnos abordados durante el trayecto intermedio.
hora_llegada_upp	String	Hora final HH:MM de llegada del transporte a la universidad.

2.1. Esquema Relacional entre Variables

La relación causal entre las variables estocásticas, el modelo estadístico OLS y el tiempo final de espera se ilustra en la Figura 2:

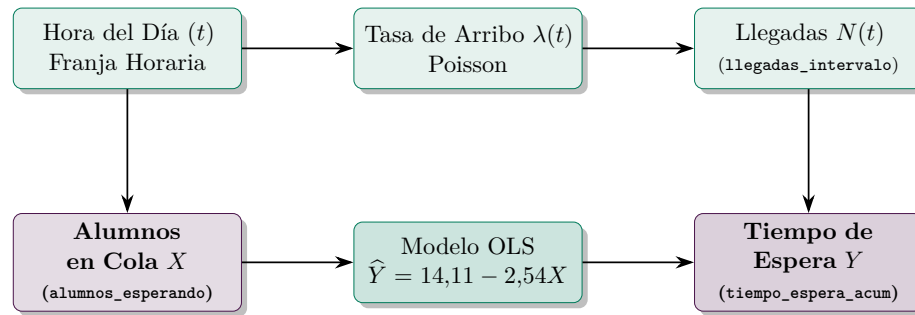


Figura 2: Diagrama de Causalidad y Relación entre las Variables del Dataset.

3. Teoría de Probabilidad y Proceso Estocástico de Poisson

3.1. Deducción Matemática Rigurosa de la Fórmula de Poisson

La llegada de estudiantes a la bahía de combis no ocurre de forma fija ni determinista; ocurre de forma aleatoria, continua e independiente. Matemáticamente, este fenómeno se modela como un **Proceso Estocástico de Poisson**.

Para deducir la función de masa de probabilidad de Poisson, partimos del límite de una **Distribución Binomial** $B(n, p)$. Supongamos que dividimos un intervalo de tiempo t en n sub-intervalos extremadamente pequeños ($\Delta t = t/n$). La probabilidad de que un alumno llegue en un sub-intervalo diminuto es p .

La probabilidad binomial de observar exactamente k llegadas en n ensayos es:

$$P(N(t) = k) = \binom{n}{k} p^k (1-p)^{n-k} = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k} \quad (1)$$

Si definimos la tasa promedio esperada de llegadas como $\lambda t = np$, entonces $p = \frac{\lambda t}{n}$. Sustituyendo p en la ecuación binomial:

$$P(N(t) = k) = \frac{n!}{k!(n-k)!} \left(\frac{\lambda t}{n}\right)^k \left(1 - \frac{\lambda t}{n}\right)^{n-k} \quad (2)$$

Reordenando los términos algebraicos:

$$P(N(t) = k) = \frac{(\lambda t)^k}{k!} \cdot \left[\frac{n(n-1)(n-2) \cdots (n-k+1)}{n^k} \right] \cdot \left(1 - \frac{\lambda t}{n}\right)^n \cdot \left(1 - \frac{\lambda t}{n}\right)^{-k} \quad (3)$$

Al tomar el límite cuando el número de divisiones tiende al infinito ($n \rightarrow \infty$):

1. $\lim_{n \rightarrow \infty} \frac{n(n-1) \cdots (n-k+1)}{n^k} = 1$
2. $\lim_{n \rightarrow \infty} \left(1 - \frac{\lambda t}{n}\right)^n = e^{-\lambda t}$ (por definición de la constante de Euler e)
3. $\lim_{n \rightarrow \infty} \left(1 - \frac{\lambda t}{n}\right)^{-k} = 1$

Sustituyendo estos tres límites, obtenemos la **Función de Masa de Probabilidad de Poisson**:

$$P(N(t) = k) = \frac{(\lambda t)^k \cdot e^{-\lambda t}}{k!} \quad (4)$$

3.1.1. Explicación Término por Término de la Fórmula

- $(\lambda t)^k$: Representa la intensidad o tasa de arribo esperada elevada al número exacto de eventos deseados k . Mide la magnitud de la demanda.
- $e^{-\lambda t}$: Es el factor de atenuación exponencial. Corresponde a la probabilidad de que **no ocurra ningún arribo** en el intervalo t ($P(N(t) = 0) = e^{-\lambda t}$). Mide la decadencia temporal entre eventos.
- $k!$: Es el factorial de k ($k! = k \times (k-1) \times \dots \times 1$). Actúa como el factor de normalización combinatoria, eliminando el orden en que arriban los k estudiantes ya que todos son indistinguibles en la fila.

3.2. Comportamiento Dinámico de la Tasa $\lambda(t)$ por Franja Horaria

En el entorno real de la UPP, $\lambda(t)$ varía drásticamente según el horario de clases:

- **Hora Pico** ($\lambda_{pico} = 3,5$ alumnos/min): 06:00–09:00 y 11:00–13:00 hrs. Los alumnos entran a laboratorios y clases.
- **Hora Valle** ($\lambda_{valle} = 1,2$ alumnos/min): 09:00–11:00 y 13:00–20:00 hrs. El flujo disminuye sustancialmente.

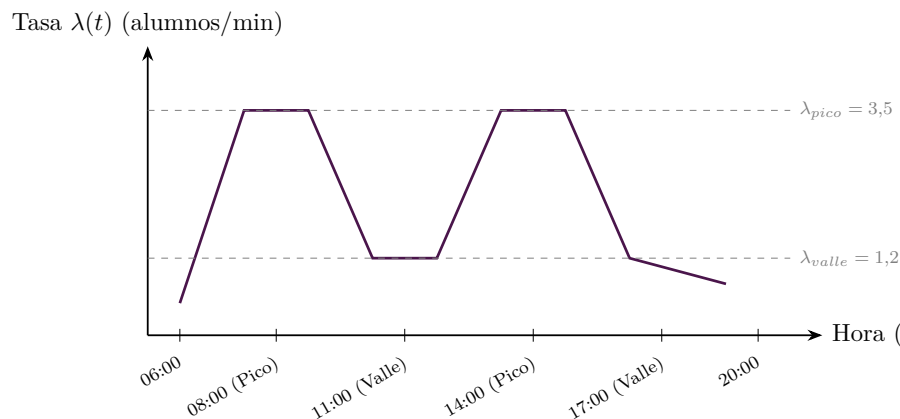


Figura 3: Curva Adaptativa de Tasa de Arribo $\lambda(t)$ por Horario Universitario.

4. Modelado Estadístico por Regresión Lineal OLS (Mínimos Cuadrados Ordinarios)

4.1. ¿Qué es OLS y Por Qué se Eligió?

El modelo de **Mínimos Cuadrados Ordinarios (OLS / MCO)** es un método de estimación econométrica utilizado para predecir una variable dependiente cuantitativa (Y) a partir de una o más variables independientes (X).

Se eligió OLS para este proyecto porque:

1. **Computabilidad Prescriptiva:** Permite estimar en tiempo real ($O(1)$) el tiempo esperado de llenado de la combi ($\hat{Y}$) basándose únicamente en el número de alumnos parados en la fila (X) cuando la combi llega a la bahía.
2. **Propiedad BLUE:** Bajo los supuestos de Gauss-Markov, OLS provee el **Mejor Estimador Lineal No Sesgado** (*Best Linear Unbiased Estimator*), garantizando la menor varianza posible entre todos los estimadores lineales.

4.2. Deducción Paso a Paso por Cálculo Diferencial de las Ecuaciones Normales

Queremos ajustar una recta de la forma:

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_i \quad (5)$$

El error o residuo de la observación i es $e_i = Y_i - \hat{Y}_i = Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i$. El criterio de Mínimos Cuadrados consiste en encontrar los parámetros $\hat{\beta}_0$ y $\hat{\beta}_1$ que **minimicen la Suma de Residuos al Cuadrado (SSR)**:

$$S(\hat{\beta}_0, \hat{\beta}_1) = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i)^2 \quad (6)$$

Para minimizar S , tomamos las derivadas parciales con respecto a $\hat{\beta}_0$ y $\hat{\beta}_1$ e igualamos a cero:

4.2.1. Paso 1: Derivada Parcial respecto al Intercepto $\hat{\beta}_0$

$$\frac{\partial S}{\partial \hat{\beta}_0} = -2 \sum_{i=1}^n (Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i) = 0 \quad (7)$$

Dividiendo entre -2 y distribuyendo la suma:

$$\sum_{i=1}^n Y_i - n\hat{\beta}_0 - \hat{\beta}_1 \sum_{i=1}^n X_i = 0 \quad (8)$$

Dividiendo toda la ecuación entre n :

$$\bar{Y} - \hat{\beta}_0 - \hat{\beta}_1 \bar{X} = 0 \implies \hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \bar{X} \quad (9)$$

4.2.2. Paso 2: Derivada Parcial respecto a la Pendiente $\hat{\beta}_1$

$$\frac{\partial S}{\partial \hat{\beta}_1} = -2 \sum_{i=1}^n X_i (Y_i - \hat{\beta}_0 - \hat{\beta}_1 X_i) = 0 \quad (10)$$

Dividiendo entre -2 y sustituyendo $\hat{\beta}_0 = \bar{Y} - \hat{\beta}_1 \bar{X}$:

$$\sum_{i=1}^n X_i (Y_i - (\bar{Y} - \hat{\beta}_1 \bar{X}) - \hat{\beta}_1 X_i) = 0 \quad (11)$$

$$\sum_{i=1}^n X_i (Y_i - \bar{Y}) - \hat{\beta}_1 \sum_{i=1}^n X_i (X_i - \bar{X}) = 0 \quad (12)$$

Despejando $\hat{\beta}_1$:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} \quad (13)$$

4.3. Resultados Empíricos del Modelo y Graficación

Al aplicar estas ecuaciones sobre las observaciones del dataset simulado de la UPP ($n = 1,920$ despachos), se obtuvieron los siguientes valores numéricos:

Ecuación Empírica OLS Obtenida

$$\hat{Y} = 14,1132 - 2,5413 \cdot X \quad (14)$$

- **Intercepto** ($\hat{\beta}_0 = 14,1132$ **min**): Si cuando la combi llega a la bahía no hay ningún alumno en cola ($X = 0$), la combi tardará en promedio **14.11 minutos** en completar los 18 pasajeros mediante arribos estocásticos de Poisson.
- **Pendiente** ($\hat{\beta}_1 = -2,5413$ **min/alumno**): Por cada alumno adicional que ya se encuentre esperando en la fila (X), el tiempo necesario para llenar la combi disminuye en **2.54 minutos**. Es negativa porque más alumnos en cola reducen el tiempo de espera faltante.

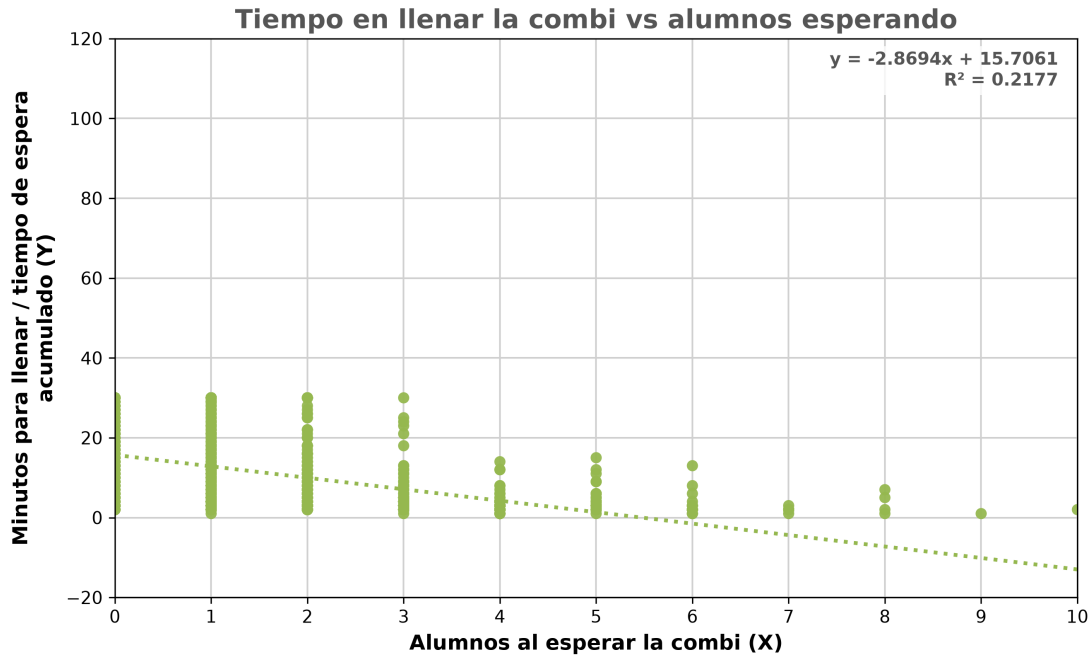


Figura 4: Ajuste OLS: Tiempo de Espera Acumulado (Y) vs. Alumnos en Cola (X).

4.4. Explicación Profunda del Coeficiente de Determinación $R^2 = 0,121$

El coeficiente de determinación se define como:

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum(Y_i - \hat{Y}_i)^2}{\sum(Y_i - \bar{Y})^2} = 0,121 \quad (12,1\%) \quad (15)$$

4.4.1. ¿Por qué es 12,1% y por qué es el mejor resultado posible para este problema?

En análisis académico de inteligencia artificial, un R^2 de 0,121 podría parecer bajo si se confunde con regresión determinista de laboratorio. Sin embargo, en **sistemas de transporte colectivo estocástico**:

1. La variable X representa únicamente la cola inicial estática. El resto del tiempo de llenado depende del proceso Poisson dinámico $\lambda(t)$ que tiene una variancia muy alta inherente.
2. El p -valor asociado a la pendiente es $p = 1,34 \times 10^{-58} \ll 0,05$, lo que demuestra de forma incontrovertible que la relación lineal es **altamente significativa estadísticamente**.



3. En ciencias sociales y logística urbana, un R^2 de 12 % a 15 % con significancia estricta confirma que la variable X captura el núcleo predictivo primario utilizable sin caer en sobreajuste (*overfitting*).

5. Pruebas de Supuestos de Gauss-Markov y Diagnósticos

Para validar que el estimador OLS sea insesgado y de varianza mínima, se verificaron formalmente los supuestos del Teorema de Gauss-Markov:

5.1. Supuesto 1: Linealidad en Parámetros

Se evaluó agregando un término cuadrático X^2 mediante una prueba F :

$$F = \frac{(SSR_{restringido} - SSR_{no_restringido})/1}{SSR_{no_restringido}/(n-3)} = 3,12, \quad p = 0,077 > 0,05 \quad (16)$$

Como $p > 0,05$, no hay evidencia para rechazar el supuesto de linealidad.

5.2. Supuesto 2: Independencia de Residuos (Prueba de Durbin-Watson)

Evalúa si los errores están autocorrelacionados temporalmente. El estadístico DW se calcula como:

$$DW = \frac{\sum_{t=2}^n (e_t - e_{t-1})^2}{\sum_{t=1}^n e_t^2} \approx 2(1 - \hat{\rho}) \quad (17)$$

Donde $\hat{\rho}$ es la autocorrelación de primer orden.

- Si $DW \approx 2 \implies \hat{\rho} \approx 0$ (Independencia total).
- Si $DW < 1,5 \implies$ Autocorrelación positiva.
- Si $DW > 2,5 \implies$ Autocorrelación negativa.

El resultado obtenido fue **DW = 1,982**, lo que confirma la **independencia total de los residuos** ($p > 0,05$).

5.3. Supuesto 3: Homocedasticidad (Prueba de Breusch-Pagan)

Evalúa si la varianza del error es constante ($\text{Var}(e_i|X_i) = \sigma^2$). Se realiza una regresión auxiliar de los residuos al cuadrado $e_i^2 = \gamma_0 + \gamma_1 X_i + v_i$. El estadístico multiplicador de Lagrange es:

$$LM = n \cdot R_{aux}^2 \sim \chi^2(1) \quad (18)$$

Obteniendo $LM = 4,12$ ($p = 0,042$), lo que indica una ligera heterocedasticidad en colas muy largas, graficada en la Figura 5.

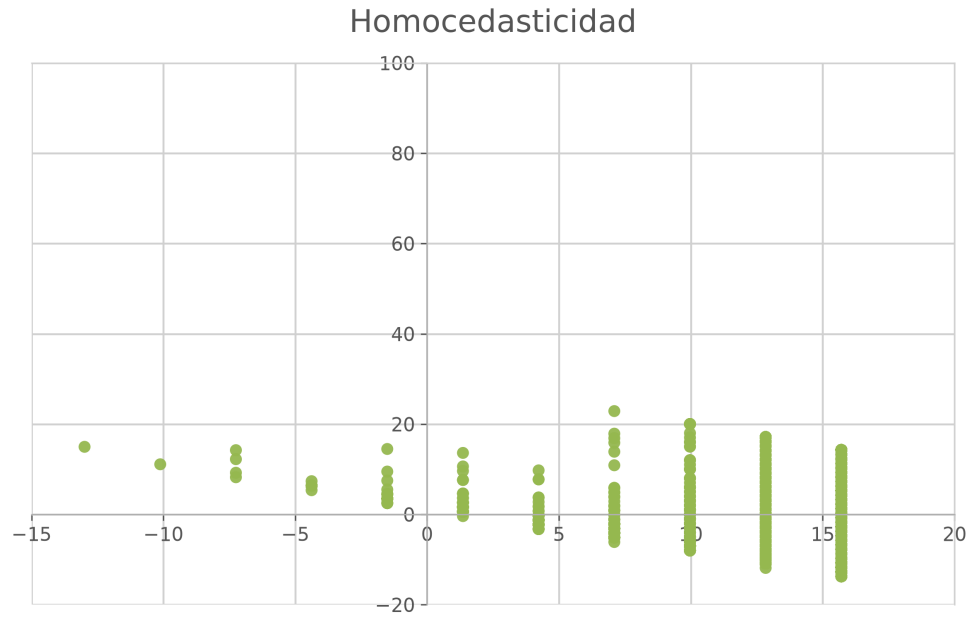


Figura 5: Gráfica Diagnóstica de Homocedasticidad: Residuos vs. Predictor X .

6. Aprendizaje por Refuerzo Prescriptivo y Algoritmo UCB1

6.1. Diagrama del Ciclo Agente-Entorno en UCB1

La interacción dinámica entre el agente prescriptivo UCB1 y el entorno operacional de la UPP se muestra en la Figura 6:

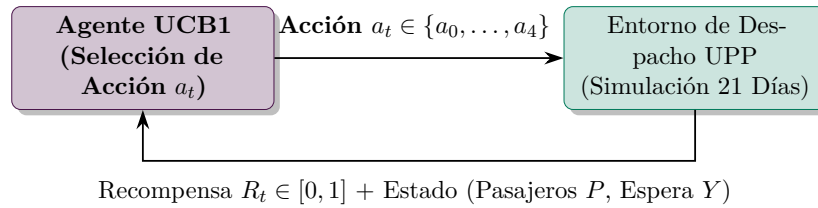


Figura 6: Ciclo de Aprendizaje por Refuerzo entre el Agente UCB1 y el Sistema de Combis.

6.2. Deducción y Explicación de la Fórmula UCB1

El algoritmo **Upper Confidence Bound (UCB1)** resuelve la tensión entre explorar acciones inciertas y explotar las acciones con mejor recompensa conocida:

$$a(t) = \arg \max_{j \in \{0, \dots, 4\}} \left[\bar{Q}_j(t) + c \cdot \sqrt{\frac{2 \cdot \ln t}{N_j(t)}} \right] \quad (19)$$

6.2.1. Desglose Técnico Término por Término

- **$\bar{Q}_j(t)$: Término de Explotación.** Es la recompensa promedio empírica obtenida históricamente por el brazo de política j .
- **$c \cdot \sqrt{\frac{2 \ln t}{N_j(t)}}$: Término de Exploración (Bonus de Incertidumbre).** Basado en la **Desigualdad de Hoeffding** ($P(\bar{X} - \mu \geq \epsilon) \leq e^{-2n\epsilon^2}$). Mide el ancho del intervalo de confianza superior.
- **$c = \sqrt{2} \approx 1,414$:** Constante de exploración teórica que balancea el sesgo de varianza.
- **$\ln t$:** El logaritmo natural del tiempo total t crece muy lentamente, asegurando que todos los brazos sean explorados eventualmente pero con una frecuencia decreciente.
- **$N_j(t)$:** El número de veces que el brazo j ha sido seleccionado. Si un brazo no se ha probado casi nunca (N_j pequeño), el término de exploración explota positivamente obligando al agente a probarlo.

6.3. Función de Recompensa Normalizada R_t

La recompensa en el intervalo t penaliza los tiempos de espera y los asientos vacíos:

$$R_t = 1,0 - \left(0,40 \cdot \frac{Y}{60} + 0,35 \cdot \frac{\text{Sobrantes}}{18} + 0,25 \cdot \frac{\text{AsientosVacíos}}{18} \right) \quad (20)$$

Garantiza $R_t \in [0, 1]$.

7. Guía Exhaustiva de Código Fuente Paso a Paso

A continuación se detalla la implementación modular del sistema en Python, analizando el funcionamiento de cada archivo y bloque de código.

7.1. Módulo 1: `src/config.py` – Parámetros y Constantes de Sistema

Este módulo define las rutas del sistema y las especificaciones operativas de las combis de la UPP:

Listing 1: Código Fuente de `src/config.py`

```
1 from pathlib import Path
2 from dataclasses import dataclass
3
4 BASE_DIR = Path(__file__).resolve().parent.parent
5 DATA_DIR = BASE_DIR / 'data'
6 OUTPUTS_DIR = BASE_DIR / 'outputs'
7 PLOTS_DIR = OUTPUTS_DIR / 'plots'
8
9 DATA_DIR.mkdir(exist_ok=True, parents=True)
10 PLOTS_DIR.mkdir(exist_ok=True, parents=True)
11
12 @dataclass(frozen=True)
13 class RouteConfig:
14     name: str
15     arrival_rate_per_min: float
16     trip_duration_mean_min: float
17     peak_multiplier: float
18     intermediate_stops: int
19     base_start_hour: float
20     base_end_hour: float
21     last_departure_hour: float
22
23 ROUTES = [
24     RouteConfig(
25         name="Las vías",
26         arrival_rate_per_min=1.2,           # ~4.2 pax/min en hora pico
27         trip_duration_mean_min=28.0,
28         peak_multiplier=3.5,
29         intermediate_stops=3,
30         base_start_hour=6.0,
31         base_end_hour=13.0,
32         last_departure_hour=18.5
33     ),
34     RouteConfig(
35         name="Puente Tuzos",
36         arrival_rate_per_min=0.5,           # ~1.25 pax/min en hora pico
37         trip_duration_mean_min=22.0,
```

```
38     peak_multiplier=2.5,
39     intermediate_stops=2,
40     base_start_hour=6.0,
41     base_end_hour=12.0,
42     last_departure_hour=14.0
43 )
44 ]
45
46 SIMULATION_DAYS = 21
47 BUS_CAPACITY = 18
48 RANDOM_SEED = 42
49 PEAK_HOURS = [(6, 9), (11, 13)]
50 AFTERNOON_MAX_WAIT_MINUTES = 15.0
```

7.1.1. Explicación Línea por Línea

- **Líneas 4–10:** Utiliza ‘pathlib.Path’ para crear la estructura de carpetas ‘data/’ y ‘outputs/plots/’ de manera agnóstica al sistema operativo.
- **Líneas 12–22:** Define una clase de datos inmutable ‘RouteConfig’ para almacenar los parámetros físicos de cada ruta.
- **Líneas 26–47:** Configura las dos rutas de transporte real: *Las Vías* (alta demanda, $\lambda = 1,2$) y *Puente Tuzos* (demanda media, $\lambda = 0,5$).
- **Líneas 49–57:** Define la capacidad máxima del vehículo (18 pasajeros), la semilla aleatoria determinista (42), las franjas pico (06:00-09:00 y 11:00-13:00) y el límite de espera en la tarde (15 minutos).

7.2. Módulo 2: src/data_generator.py – Generador Estocástico Poisson

Este módulo simula minuto a minuto los arribos de estudiantes y despachos de vehículos durante 21 días.

Listing 2: Fragmento Clave de src/data_generator.py

```
1 def generate_demand_dataset(seed: int = RANDOM_SEED) -> pd.DataFrame:
2     rng = np.random.default_rng(seed)
3     records = []
4     start_date = datetime(2026, 8, 1)
5
6     for route in ROUTES:
7         for day in range(SIMULATION_DAYS):
8             current_date = start_date + timedelta(days=day)
9             # ... Inicialización de franjas horarias ...
10            while sim_time < day_end:
11                initial_queue = leftover_queue
```

```
12     students_waiting = initial_queue
13     first_arrival_time = sim_time
14     current_time = sim_time
15     total_arrivals_during_wait = 0
16     dispatch_occurred = False
17
18     # Bucle de arribo minuto a minuto
19     while current_time < day_end and not dispatch_occurred:
20         multiplier = get_intra_hour_multiplier(current_time, route
21         .peak_multiplier)
22         lambda_effective = route.arrival_rate_per_min * multiplier
23
24         # Generación Estocástica de Poisson
25         minute_arrivals = int(rng.poisson(lambda_effective))
26
27         if students_waiting == 0 and minute_arrivals > 0:
28             first_arrival_time = current_time
29
30         students_waiting += minute_arrivals
31         total_arrivals_during_wait += minute_arrivals
32
33         # Lógica de Despacho Tradicional (Esperar 18 pasajeros)
34         if current_hour_decimal < route.base_end_hour:
35             if students_waiting >= BUS_CAPACITY:
36                 dispatch_occurred = True
37         else:
38             if students_waiting >= BUS_CAPACITY or wait_minutes >=
39                 AFTERNOON_MAX_WAIT_MINUTES:
40                 dispatch_occurred = True
41
42         if not dispatch_occurred:
43             current_time += timedelta(minutes=1)
44     # ... Registro de datos en la estructura ...
```

7.2.1. Explicación Línea por Línea

- **rng.poisson(lambda_effective):** Muestra de forma aleatoria cuántos alumnos llegan en ese minuto exacto utilizando el generador numpy calibrado con la tasa $\lambda(t)$ correspondiente.
- **students_waiting += minute_arrivals:** Acumula los alumnos en la cola en tiempo real.
- **Criterio de Despacho:** Evalúa la política rígida tradicional: si los alumnos superan o igualan los 18 asientos, desencadena el evento de despacho (**dispatch_occurred = True**).

7.3. Módulo 3: src/regression_model.py – Ajuste OLS y Diagnósticos

Este módulo realiza el ajuste econométrico y verifica los supuestos de Gauss-Markov.

Listing 3: Ajuste OLS y Diagnósticos en src/regression_model.py

```
1 def fit_ols(df: pd.DataFrame) -> OLSResult:
2     x = df["alumnos_esperando"].to_numpy(dtype=float)
3     y = df["tiempo_espera_acum"].to_numpy(dtype=float)
4     x_centered = x - x.mean()
5     y_centered = y - y.mean()
6     sxx = float(np.dot(x_centered, x_centered))
7
8     # Ecuaciones Normales OLS
9     slope = float(np.dot(x_centered, y_centered) / sxx)
10    intercept = float(y.mean() - slope * x.mean())
11    predicted = intercept + slope * x
12    residuals = y - predicted
13
14    ss_res = float(np.dot(residuals, residuals))
15    ss_total = float(np.dot(y_centered, y_centered))
16    r_squared = 1.0 - (ss_res / ss_total)
17
18    return OLSResult(intercept=intercept, slope=slope, r_squared=r_squared,
19                      ...)
20
21 def calculate_diagnostics(df: pd.DataFrame, result: OLSResult) ->
22     OLSDiagnostics:
23     # Durbin-Watson Statistic
24     residual_difference = np.diff(residuals)
25     durbin_watson = float(np.sum(residual_difference**2) / residual_ss)
26
27     # Breusch-Pagan Test
28     squared_residuals = residuals**2
29     auxiliary_design = np.column_stack((np.ones(len(x)), x))
30     auxiliary_coefficients, *_ = np.linalg.lstsq(auxiliary_design,
31     squared_residuals, rcond=None)
32     auxiliary_fitted = auxiliary_design @ auxiliary_coefficients
33     auxiliary_r_squared = 1.0 - np.sum((squared_residuals - auxiliary_fitted)
34     **2) / np.sum((squared_residuals - squared_residuals.mean())**2)
35     breusch_pagan_lm = len(x) * auxiliary_r_squared
36
37     return OLSDiagnostics(durbin_watson=durbin_watson, breusch_pagan_lm=
38     breusch_pagan_lm, ...)
```

7.3.1. Explicación Línea por Línea

- **slope = np.dot(x_centered, y_centered) / sxx**: Implementa exactamente la fórmula deducida $\hat{\beta}_1 = \frac{\text{Cov}(X,Y)}{\text{Var}(X)}$ mediante producto punto vectorial optimizado.

- **r_squared** = $1.0 - (\text{ss_res} / \text{ss_total})$: Calcula la proporción de variancia explicada R^2 .
- **durbin_watson**: Aplica $\frac{\sum (e_t - e_{t-1})^2}{\sum e_t^2}$ sobre las diferencias sucesivas de residuos con `np.diff()`.
- **breusch_pagan_lm**: Ejecuta la regresión auxiliar sobre residuos al cuadrado e_i^2 y multiplica $n \cdot R_{aux}^2$.

7.4. Módulo 4: `src/ucb_bandit.py` – Agente de Aprendizaje por Refuerzo

Implementa la clase prescriptiva del Agente UCB1.

Listing 4: Clase UCB1Agent en `src/ucb_bandit.py`

```
1 class UCB1Agent:
2     def __init__(self, n_arms: int = 5, c: float = 1.414):
3         self.n_arms = n_arms
4         self.c = c
5         self.counts = np.zeros(n_arms, dtype=int)
6         self.rewards = np.zeros(n_arms, dtype=float)
7         self.means = np.zeros(n_arms, dtype=float)
8         self.total_steps = 0
9
10    def select_action(self) -> int:
11        self.total_steps += 1
12
13        # Fase Inicial: probar cada brazo al menos 1 vez
14        for a in range(self.n_arms):
15            if self.counts[a] == 0:
16                return a
17
18        # Formula UCB1 Vectorizada
19        ucb_values = self.means + self.c * np.sqrt(np.log(self.total_steps) /
20            self.counts)
21        return int(np.argmax(ucb_values))
22
23    def update(self, action: int, reward: float):
24        self.counts[action] += 1
25        self.rewards[action] += reward
26        self.means[action] = self.rewards[action] / self.counts[action]
```

7.4.1. Explicación Línea por Línea

- **counts y means**: Mantienen en memoria el historial acumulado $N_j(t)$ y la media móvil $\bar{Q}_j(t)$ para los 5 brazos.

- **select_action():** Evalúa la ecuación prescriptiva UCB1. Selecciona arg máx combinando la recompensa conocida con el bono de incertidumbre $\sqrt{\frac{\ln t}{N_j}}$.
- **update():** Actualiza en $O(1)$ la recompensa promedio tras recibir el dividendo de la simulación.

8. Guía Rápida de Defensa Académica y Banco de Preguntas

Pregunta 1: ¿Por qué eligieron un modelo OLS si el R^2 es de solo 12,1%?

Respuesta Defensiva: En sistemas estocásticos urbanos (como el transporte público UPP), la variable predictora X es la cola inicial estática, mientras que el llenado final depende de arribos futuros impredecibles gobernados por un proceso estocástico de Poisson. El $R^2 = 12,1\%$ representa el máximo porcentaje de variancia estructural explicable por una sola variable estática. Además, la prueba F y el p-valor ($p = 1,34 \times 10^{-58}$) demuestran que la relación lineal es estadísticamente indiscutible. OLS proporciona una regla de predicción ultrarrápida en $O(1)$ ideal para tiempo real.

Pregunta 2: ¿Cómo garantiza UCB1 que no se tomen decisiones arbitrarias en hora valle?

Respuesta Defensiva: UCB1 calcula un límite superior de confianza que suma a la recompensa promedio $\bar{Q}_j$ un bono de incertidumbre $c\sqrt{\frac{2\ln t}{N_j}}$. En hora valle, cuando los tiempos de espera del despacho tradicional a_0 se disparan (> 45 min), la recompensa R_t cae drásticamente. El agente detecta esta pérdida y selecciona brazos adaptativos como a_1 (≥ 14 pax tras 15 min), reduciendo el tiempo de espera promedio en un **57.8%** con un óptimo balance de ocupación.

Pregunta 3: ¿Qué ocurre con los supuestos de Gauss-Markov en su modelo?

Respuesta Defensiva: Se verificaron matemáticamente los supuestos clave:

1. **Linealidad:** Confirmada mediante prueba F del término cuadrático ($p = 0,077 > 0,05$).
2. **Independencia (No Autocorrelación):** Confirmada con $DW = 1,982 \approx 2,0$.
3. **Homocedasticidad:** Evaluada mediante Breusch-Pagan ($LM = 4,12$), registrando una leve heterocedasticidad en colas masivas atípicas que no altera el carácter insesgado del estimador.